

```

In [1]: # =====
# CELL 1: CONSTANTS & CONFIGURATION
# =====

import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import unicodedata
import os

VALID_LEAGUES = ['Premier League', 'Ligue 1', 'Bundesliga', 'Serie A', 'La Liga']

LEAGUE_MAP = {
    'Ligue 1': 0, 'Bundesliga': 1, 'Serie A': 2, 'La Liga': 3, 'Premier League': 4
}

# Position-specific stat lists
# FWD and MID share the core 4 (mirrors Paper 2), with extras per position
STATS_FWD = ['KP', 'PrgP', 'PrgC', 'SCA', 'Goals', 'Assists', 'xG', 'npxG', 'GCA', 'PPA']
STATS_MID = ['KP', 'PrgP', 'PrgC', 'SCA', 'GCA', 'PPA', 'PassPct', 'PrgDist']
STATS_DEF = ['TkLW', 'Int', 'Clr', 'Blocks', 'PrgP', 'PassPct']

POS_STATS_MAP = {
    'FWD': STATS_FWD,
    'MID': STATS_MID,
    'DEF': STATS_DEF,
}

# Union of all metrics – used for loading & column creation
ALL_METRICS = sorted(set(STATS_FWD + STATS_MID + STATS_DEF))

# Seasons
HISTORICAL_SEASONS = [
    '2017_2018', '2018_2019', '2019_2020',
    '2020_2021', '2021_2022', '2022_2023', '2023_2024',
]
MODERN_SEASONS = ['2024_2025']
ALL_SEASONS = HISTORICAL_SEASONS + MODERN_SEASONS

# All consecutive pairs (t → t+1)
ALL_PAIRS = [(ALL_SEASONS[i], ALL_SEASONS[i+1]) for i in range(len(ALL_SEASONS) - 1)]

MIN_90S = 5.0
POS_BUCKETS = ['FWD', 'MID', 'DEF']

print("=" * 70)
print("METRIC STABILITY – DATA PREPARATION")
print("=" * 70)
print(f"\n Seasons           : {len(ALL_SEASONS)}")
print(f" Season pairs       : {len(ALL_PAIRS)}")
print(f" MIN_90S filter    : {MIN_90S}")
print(f" Leagues           : {' , '.join(VALID_LEAGUES)}")

```

```

print(f"\n Stats per position group:")
for pos, stats in POS_STATS_MAP.items():
    print(f"    {pos} ({len(stats)}): {stats}")
print(f"\n Union of all metrics ({len(ALL_METRICS)}): {ALL_METRICS}")

```

===== METRIC STABILITY – DATA PREPARATION =====

```

Seasons      : 8
Season pairs : 7
MIN_90S filter : 5.0
Leagues      : Premier League, Ligue 1, Bundesliga, Serie A, La Liga

```

Stats per position group:

```

FWD (10): ['KP', 'PrgP', 'PrgC', 'SCA', 'Goals', 'Assists', 'xG', 'npG', 'GCA', 'PPA']
MID (8):  ['KP', 'PrgP', 'PrgC', 'SCA', 'GCA', 'PPA', 'PassPct', 'PrgDist']
DEF (6):  ['TkLW', 'Int', 'Clr', 'Blocks', 'PrgP', 'PassPct']

```

```

Union of all metrics (16): ['Assists', 'Blocks', 'Clr', 'GCA', 'Goals', 'Int', 'KP', 'PPA', 'PassPct', 'PrgC', 'PrgDist', 'PrgP', 'SCA', 'TkLW', 'npG', 'xG']

```

```

In [2]: # =====
# CELL 2: DATA LOADING HELPERS
# =====

def normalize_name(x: str) -> str:
    x = unicodedata.normalize('NFKD', str(x))
    x = x.encode('ascii', 'ignore').decode('ascii')
    return x.replace(' ', '').lower()

def bucket_position(pos: str) -> str:
    primary = str(pos).upper().split(',')[0].strip()
    if 'D' in primary: return 'DEF'
    elif 'M' in primary: return 'MID'
    elif 'F' in primary: return 'FWD'
    return 'OTHER'

def standardize_league(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df['Comp'] = (
        df['Comp'].astype(str)
        .str.replace(r'^[a-z]{2,3}\s+', '', regex=True)
        .str.strip()
    )
    df['Comp'] = df['Comp'].replace({'eng Premier League': 'Premier League'})
    df['Pos_Bucket'] = df['Pos'].apply(bucket_position)
    return df

def load_historical(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    df.columns = [c.strip() for c in df.columns]
    df = df.rename(columns={

```

```

        'player': 'Player', 'squad': 'Squad',
        'comp': 'Comp',      'pos': 'Pos', 'age': 'Age',
    })
    # FIX: 'Avg Mins per Match' is total season minutes in this dataset
    df['90s'] = pd.to_numeric(df.get('Avg Mins per Match', 0), errors='coerce').fillna(0) /
    df['Player_Key'] = df['Player'].apply(normalize_name)
    df = standardize_league(df)
    return df

def load_modern(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    df.columns = [c.strip() for c in df.columns]
    if '90s' not in df.columns and 'Min' in df.columns:
        df['90s'] = pd.to_numeric(df['Min'], errors='coerce').fillna(0) / 90.0
    df['Player_Key'] = df['Player'].apply(normalize_name)
    df = standardize_league(df)
    return df

print("✓ Loading helpers defined")

```

✓ Loading helpers defined

```

In [3]: # =====
# CELL 3: COLUMN MAPPING HELPERS
# =====

def add_att_stats(df: pd.DataFrame, is_historical: bool) -> pd.DataFrame:
    df = df.copy()
    if is_historical:
        df['KP'] = pd.to_numeric(df.get('Key passes', 0), errors='coerce').fillna(0)
        df['PrgP'] = pd.to_numeric(df.get('Progressive Passes', 0), errors='coerce').fillna(0)
        df['PrgC'] = pd.to_numeric(df.get('Progressive Carries', 0), errors='coerce').fillna(0)
        # SCA/GCA stored as per-90 in historical → convert to raw counts
        df['SCA'] = (
            pd.to_numeric(df.get('Shot creating actions p 90', 0), errors='coerce').fillna(0)
            * df['90s']
        )
        df['GCA'] = (
            pd.to_numeric(df.get('Goal creating actions p 90', 0), errors='coerce').fillna(0)
            * df['90s']
        )
        df['Goals'] = pd.to_numeric(df.get('Goals', 0), errors='coerce')
        df['Assists'] = pd.to_numeric(df.get('Assists', 0), errors='coerce')
        df['xG'] = pd.to_numeric(df.get('Expected Goals', 0), errors='coerce')
        df['npxG'] = pd.to_numeric(df.get('Exp NPG', 0), errors='coerce')
        df['PPA'] = pd.to_numeric(df.get('Passes into penalty area', 0), errors='coerce')
        df['PassPct'] = pd.to_numeric(df.get('Pass completion %', 0), errors='coerce')
        df['PrgDist'] = pd.to_numeric(df.get('Progressive passes distance', 0), errors='coerce')
    else:
        for col in ['KP', 'PrgP', 'PrgC']:
            df[col] = pd.to_numeric(df.get(col, 0), errors='coerce').fillna(0)
        if 'SCA' in df.columns:
            df['SCA'] = pd.to_numeric(df['SCA'], errors='coerce').fillna(0)
        elif 'SCA90' in df.columns:

```

```

        df['SCA'] = pd.to_numeric(df['SCA90'], errors='coerce').fillna(0) * df['90s']
    else:
        df['SCA'] = 0.0
    if 'GCA' in df.columns:
        df['GCA'] = pd.to_numeric(df['GCA'], errors='coerce').fillna(0)
    elif 'GCA90' in df.columns:
        df['GCA'] = pd.to_numeric(df['GCA90'], errors='coerce').fillna(0) * df['90s']
    else:
        df['GCA'] = 0.0
    df['Goals'] = pd.to_numeric(df.get('GLs', 0), errors='coerce').fillna(0)
    df['Assists'] = pd.to_numeric(df.get('Ast', 0), errors='coerce').fillna(0)
    df['xG'] = pd.to_numeric(df.get('xG', 0), errors='coerce').fillna(0)
    df['npxG'] = pd.to_numeric(df.get('npxG', 0), errors='coerce').fillna(0)
    df['PPA'] = pd.to_numeric(df.get('PPA', 0), errors='coerce').fillna(0)
    df['PassPct'] = pd.to_numeric(df.get('Cmp%', 0), errors='coerce').fillna(0)
    df['PrgDist'] = pd.to_numeric(df.get('PrgDist', 0), errors='coerce').fillna(0)
    return df

def add_def_stats(df: pd.DataFrame, is_historical: bool) -> pd.DataFrame:
    df = df.copy()
    if is_historical:
        df = df.rename(columns={
            'Tackles Won': 'TklW', 'Interceptions': 'Int', 'Clearances': 'Clr',
        })
        shots = pd.to_numeric(df.get('Shots blocked', 0), errors='coerce').fillna(0)
        passes = pd.to_numeric(df.get('Passes blocked', 0), errors='coerce').fillna(0)
        df['Blocks'] = shots + passes
        for col in ['TklW', 'Int', 'Clr']:
            df[col] = pd.to_numeric(df.get(col, 0), errors='coerce').fillna(0)
    else:
        for col in ['TklW', 'Int', 'Clr']:
            df[col] = pd.to_numeric(df.get(col, 0), errors='coerce').fillna(0)
        if 'Blocks_stats_defense' in df.columns:
            df['Blocks'] = pd.to_numeric(df['Blocks_stats_defense'], errors='coerce').fillna(0)
        else:
            df['Blocks'] = pd.to_numeric(df.get('Blocks', 0), errors='coerce').fillna(0)
    return df

def add_per90_stats(df: pd.DataFrame) -> pd.DataFrame:
    """Add per-90 versions of all metrics (for sensitivity checks)."""
    df = df.copy()
    safe_90s = df['90s'].replace(0, np.nan)
    for col in ALL_METRICS:
        if col in df.columns and col != 'PassPct': # PassPct is already a rate
            df[f'{col}_p90'] = df[col] / safe_90s
    return df

print("✓ Column mapping helpers defined")

```

✓ Column mapping helpers defined

```

In [4]: # =====
        # CELL 4: LOAD ALL SEASONS

```

```
# =====
print("Loading all seasons...\n")

raw_data = {}

for s in HISTORICAL_SEASONS:
    path = f'players_data-{s}.csv'
    if not os.path.exists(path):
        print(f" ⚠ File not found: {path} - skipping")
        continue
    df = load_historical(path)
    df = add_att_stats(df, is_historical=True)
    df = add_def_stats(df, is_historical=True)
    df = add_per90_stats(df)
    raw_data[s] = df
    print(f" {s}: {len(df)} rows, 90s mean={df['90s'].mean():.2f}")

for s in MODERN_SEASONS:
    path = f'players_data-{s}.csv'
    if not os.path.exists(path):
        print(f" ⚠ File not found: {path} - skipping")
        continue
    df = load_modern(path)
    df = add_att_stats(df, is_historical=False)
    df = add_def_stats(df, is_historical=False)
    df = add_per90_stats(df)
    raw_data[s] = df
    print(f" {s} (modern): {len(df)} rows, 90s mean={df['90s'].mean():.2f}")

print(f"\n✓ Loaded {len(raw_data)} seasons")
```

Loading all seasons...

```
2017_2018: 2549 rows, 90s mean=15.71
2018_2019: 2503 rows, 90s mean=15.99
2019_2020: 2558 rows, 90s mean=14.77
2020_2021: 2634 rows, 90s mean=15.18
2021_2022: 2689 rows, 90s mean=14.87
2022_2023: 2687 rows, 90s mean=14.88
2023_2024: 2623 rows, 90s mean=14.62
2024_2025 (modern): 2854 rows, 90s mean=13.46
```

✓ Loaded 8 seasons

```
In [5]: # =====
# CELL 5: FILTER BY VALID LEAGUES & MINIMUM PLAYING TIME
# =====
# Save panel at MIN_90S_SAVE = 3.0 so the analysis script can sweep
# thresholds 3.0 → 7.0 meaningfully. The analysis script applies
# the chosen MIN_90S = 5.0 filter before computing stability.

MIN_90S_SAVE = 3.0 # lower bound for panel – analysis script filters further

print("Applying league filter and minimum playing time filter...\n")
```

```

filtered_data = {}

for s, df in raw_data.items():
    n_before = len(df)
    df_f = df[
        (df['90s'] >= MIN_90S_SAVE) &
        (df['Comp'].isin(VALID_LEAGUES)) &
        (df['Pos_Bucket'].isin(POS_BUCKETS))
    ].copy()
    filtered_data[s] = df_f
    print(f" {s}: {n_before} → {len(df_f)} rows "
          f"(removed {n_before - len(df_f)}, "
          f"kept {len(df_f)/n_before*100:.1f}%)")

print(f"\n Total player-seasons after filtering: "
      f"{sum(len(d) for d in filtered_data.values())}")
print(f"\n Note: filtered at MIN_90S_SAVE={MIN_90S_SAVE} - "
      f"analysis script applies chosen MIN_90S={MIN_90S} before stability computation")

```

Applying league filter and minimum playing time filter...

```

2017_2018: 2549 → 1997 rows (removed 552, kept 78.3%)
2018_2019: 2503 → 1966 rows (removed 537, kept 78.5%)
2019_2020: 2558 → 1980 rows (removed 578, kept 77.4%)
2020_2021: 2634 → 2034 rows (removed 600, kept 77.2%)
2021_2022: 2689 → 2070 rows (removed 619, kept 77.0%)
2022_2023: 2687 → 2054 rows (removed 633, kept 76.4%)
2023_2024: 2623 → 2009 rows (removed 614, kept 76.6%)
2024_2025: 2854 → 2044 rows (removed 810, kept 71.6%)

```

Total player-seasons after filtering: 16154

Note: filtered at MIN_90S_SAVE=3.0 – analysis script applies chosen MIN_90S=5.0 before stability computation

```

In [6]: # =====
# CELL 6: CONSTRUCT CONSECUTIVE SEASON PAIRS
# =====
# Design:
# - Same league in BOTH seasons (within-league stability)
# - Multi-club stints: keep row with most 90s
# - Both seasons must meet MIN_90S (already filtered above)

print("Constructing consecutive season pairs...\n")

KEEP_COLS = (
    ['Player_Key', 'Player', 'Squad', 'Comp', 'Pos', 'Pos_Bucket', 'Age', '90s']
    + ALL_METRICS
    + [f'{m}_p90' for m in ALL_METRICS if m != 'PassPct']
)

def deduplicate_season(df: pd.DataFrame) -> pd.DataFrame:
    """Keep the row with most 90s if a player has multi-club stints."""
    return (
        df.sort_values('90s', ascending=False)
        .drop_duplicates(subset=['Player_Key', 'Comp'], keep='first')
    )

```

```

    )

pair_dfs = []

for (season_t, season_t1) in ALL_PAIRS:
    if season_t not in filtered_data or season_t1 not in filtered_data:
        print(f" ⚠ Skipping {season_t} → {season_t1} (missing data)")
        continue

    df_t = deduplicate_season(filtered_data[season_t])
    df_t1 = deduplicate_season(filtered_data[season_t1])

    cols_t = [c for c in KEEP_COLS if c in df_t.columns]
    cols_t1 = [c for c in KEEP_COLS if c in df_t1.columns]

    df_t = df_t[cols_t].rename(columns={c: f'{c}_t' for c in cols_t if c != 'Player_Key'})
    df_t1 = df_t1[cols_t1].rename(columns={c: f'{c}_t1' for c in cols_t1 if c != 'Player_Ke'})

    merged = pd.merge(df_t, df_t1, on='Player_Key', how='inner')

    # Same league only
    merged = merged[merged['Comp_t'] == merged['Comp_t1']].copy()

    merged['Season_t'] = season_t
    merged['Season_t1'] = season_t1
    merged['Pair'] = f'{season_t}_to_{season_t1}'

    pair_dfs.append(merged)
    print(f" {season_t} → {season_t1}: {len(merged)} same-league pairs")

print(f"\n✓ All pairs constructed")

```

Constructing consecutive season pairs...

```

2017_2018 → 2018_2019: 1286 same-league pairs
2018_2019 → 2019_2020: 1262 same-league pairs
2019_2020 → 2020_2021: 1345 same-league pairs
2020_2021 → 2021_2022: 1345 same-league pairs
2021_2022 → 2022_2023: 1281 same-league pairs
2022_2023 → 2023_2024: 1237 same-league pairs
2023_2024 → 2024_2025: 1237 same-league pairs

```

✓ All pairs constructed

```

In [7]: # =====
# CELL 7: COMBINE, ADD DERIVED VARIABLES & SAVE
# =====

panel = pd.concat(pair_dfs, ignore_index=True)

print(f" Total observations : {len(panel)}")
print(f" Unique players      : {panel['Player_Key'].nunique()}")
print(f" Season pairs          : {panel['Pair'].nunique()}")

# Age group (for extension analysis)
def age_group(age_str) -> str:

```

```

try:
    age = int(str(age_str).split('-')[0])
    if age < 23:    return 'Young (U23)'
    elif age <= 29: return 'Prime (23-29)'
    else:         return 'Veteran (30+)'
except:
    return 'Unknown'

panel['Age_Group'] = panel['Age_t'].apply(age_group)
panel['Age_Numeric'] = panel['Age_t'].apply(
    lambda x: int(str(x).split('-')[0]) if pd.notna(x) else np.nan
)
panel['League_ID'] = panel['Comp_t'].map(LEAGUE_MAP)

print(f"\n Position breakdown:")
for pos in POS_BUCKETS:
    n = (panel['Pos_Bucket_t'] == pos).sum()
    print(f"    {pos}: {n} ({n/len(panel)*100:.1f}%)")

print(f"\n Age group breakdown:")
for grp, cnt in panel['Age_Group'].value_counts().items():
    print(f"    {grp}: {cnt} ({cnt/len(panel)*100:.1f}%)")

# Save
panel.to_csv('metric_stability_panel.csv', index=False)
print(f"\n✓ Saved: metric_stability_panel.csv  ({len(panel)} rows × {len(panel.columns)} c
print("\nNext → metric_stability_analysis.py")

```

Total observations : 8993
 Unique players : 3061
 Season pairs : 7

Position breakdown:
 FWD: 2162 (24.0%)
 MID: 3096 (34.4%)
 DEF: 3735 (41.5%)

Age group breakdown:
 Prime (23-29): 5210 (57.9%)
 Young (U23): 2272 (25.3%)
 Veteran (30+): 1511 (16.8%)

✓ Saved: metric_stability_panel.csv (8993 rows × 83 cols)

Next → metric_stability_analysis.py